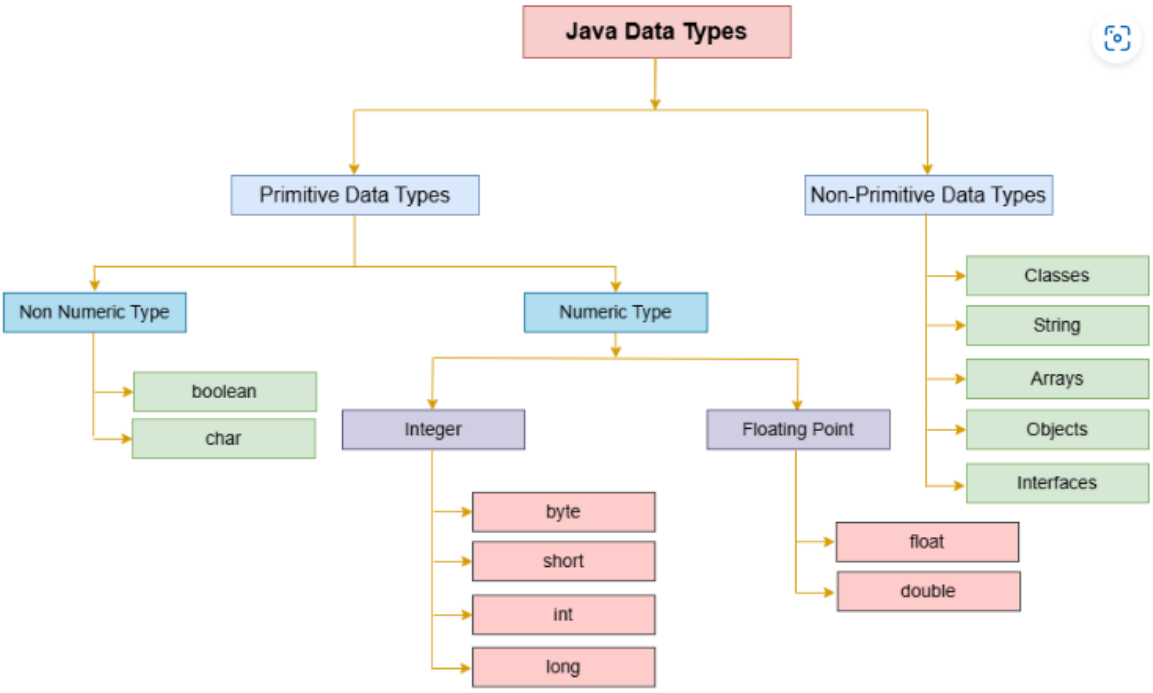
Data Types in Java

In programming languages, data types specify the different sizes and values that can be stored in the variable or constants. Each data type is predefined, which makes Java a statically and strongly typed language. There are the following two types of data types in Java.

- Primitive Data Types: The primitive data types include boolean, char, byte, short, int, long, float and double.
- Non-Primitive Data Types: The non-primitive data types include Classes, Interfaces, String, and Arrays



In Java, primitive data types are the building blocks of data manipulation. These are the basic data types.

In Java, there are mainly eight primitive data types which are as follows.

- boolean data type
- char data type
- byte data type
- short data type
- int data type
- long data type
- float data type
- double data type

1. Boolean Data Type
In Java, the boolean data type represents a single bit of information with two possible states: true or false. The size of the Boolean data type is 1 byte (8 bits).

It is used to store the result of logical expressions or conditions. Unlike other primitive data types like int or double, boolean does not have a specific size or range. It is typically implemented as a single bit, although the exact implementation may vary across platforms.

Syntax:
boolean flag;
Example
Boolean a = false;
Boolean b = true;
2. Byte Data Type

The byte data type in Java is a primitive data type that represents an 8-bits signed two's complement integer. It has a range of values from -128 to 127. Its default value is 0.

The byte data type is commonly used when working with raw binary data or when memory conservation is a concern, as it occupies less memory than larger integer types like int or long.
Syntax:
byte size;
Example
byte a = 10;
byte b = -20;
3. Short Data Type
The short data type in Java is a primitive data type that represents a 16-bits signed two-complement integer. Its range of values is -32,768 to 32,767.

Similar to the byte data type, short is used when memory conservation is a concern, but more precision than byte is required. Its default value is 0.
Syntax:
short var;
Example
short a = 10000;
short b = -5000;
4. int Data Type
The int data type in Java is a primitive data type that represents a 32-bits signed two's complement integer. It has a range of values from -2,147,483,648 to 2,147,483,647.

The int data type is one of the most commonly used data types. It is typically used to store whole numbers without decimal points. Its default value is 0.
Syntax:

int myInt = 54;
In Java, int variables are declared using the int keyword. For example, int myInt = 54 ; declares an int variable named myInt and initializes it with the value 54. int variables can be used in mathematical expressions, assigned to other int variables, and used in conditional statements.

Remember: In Java SE 8 and later versions, we can use the int data type to represent an unsigned 32-bit integer. It has a value in the range [0, 232-1]. Use the Integer class to use the int data type as an unsigned integer.
Example
int a = 100000;
int b = -200000;
5. long Data Type
The long data type in Java is a primitive data type that represents a 64-bits signed two's complement integer. It has a wider range of values than int, ranging from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807. Its default value is 0.0L or 0.0l.

The long data type is used when the int data type is not large enough to hold the desired value or when a larger range of integer values is needed.
Syntax:
long num = 15000000000L;
long num = 9,223,372,036,854,775l
The long data type is commonly used in applications where large integer values are required, such as in scientific computations, financial applications, and systems programming. It provides greater precision and a larger range than int, making it suitable for scenarios where int is insufficient.
long a = 5000000L;
long b = -6000000L;
6. float Data Type
The float data type in Java is a primitive data type that represents single-precision 32-bits IEEE 754 floating-point numbers. It can represent a wide range of decimal values, but it is not suitable for precise values such as currency. Its default value is 0.0f or 0.0F.

The float data type is useful for applications where a higher range of values is needed and precision is not critical.
Syntax:
float num = 67;
Example
float f = 234.5f;
7. double Data Type
The double data type in Java is a primitive data type that represents double-precision 64-bits IEEE 754 floating-point numbers. Its default value is 0.0. It provides a wider range of values and greater precision compared to the float data type, which makes it suitable for applications where accurate representation of decimal values is required.

Syntax:
double num = 75.658d;
Example
double d = 12.3;
8. char Data Type
The char data type in Java is a primitive data type that represents a single 16-bits Unicode character. It can store any character from the Unicode character set, which allows Java to support the internationalisation and representation of characters from various languages and writing systems.

Syntax:
char alphabets = 'J';
char a = 60, b = 61, c = 62;
Example
char c = 'A';

Type	Default Values	Size	Range of Values	Example
boolean	false	8 bits	true, false	true, false, 0, 1
byte	0	8 bits	-128 to 127	-
char	\u0000	16 bits	Characters Representation of ASCII values 0 to 255	'z', '\u0041', '\11', '\\', '\', '\n'
short	0	16 bits	-32,768 to 32,767	-
int	0	32 bits	-2,147,483,648 to 2,147,483,647	-5,-1,0,6,8

long	0	64 bits	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	-7L,-2L,0L,1L,2L
float	0.0f	32 bits	up to 7 decimal digits	3.14f, 89.09F, -9.3F
double	0.0	64 bits	up to 16 decimal digits	6.98765e300d , -567899e-600d , 2e2d

Java Primitive Data Type Example
Example

```
public class Main
{
   public static void main(String args[])
   {
      boolean flag = true;
      char ch = 'z';
      int num = 1234;
      byte size = 2;
      short srt = 78;
      double value = 2.4546778;
      float temp = 3.8f;
      long val = 1888889;
      System.out.println("boolean: " + flag);
      System.out.println("char: " + ch);
      System.out.println("integer: " + num);
      System.out.println("byte: " + size);
      System.out.println("short: " + srt);
      System.out.println("float: " + value);
      System.out.println("double: " + temp);
      System.out.println("long: " + val);
   }
}
```

Non-Primitive Data Types in Java
In Java, non-primitive data types are also known as reference data types. It is used to store complex objects rather than simple values. Reference data types store references or memory

addresses that point to the location of the object in memory. This distinction is important because it affects how these data types are stored, passed, and manipulated in Java programs.

1. Class

   One common non-primitive data type in Java is the class. Classes are used to create objects, which are instances of the class. A class defines the properties and behaviours of objects, including variables (fields) and methods.

   For example, you might create a Person class to represent a person, with variables for the person's name, age, and address, and methods to set and get these values.

   Syntax:

   ```
   class Main
   {
   //code
   }
   ```

2. Interface

   Interfaces are another important non-primitive data type in Java. An interface defines a contract for what a class implementing the interface must provide without specifying how it should be implemented. Interfaces are used to achieve abstraction and multiple inheritance in Java, allowing classes to be more flexible and reusable.

   Syntax:

   ```
   // interface
   interface Shape {
     public void draw(); // interface method (does not have a body)
     public void color(); // interface method (does not have a body)
   }
   ```

3. Arrays

   Arrays are a fundamental non-primitive data type in Java that allows you to store multiple values of the same type in a single variable. Arrays have a fixed size, which is specified when the array is created and can be accessed using an index. Arrays are commonly used to store lists of values or to represent matrices and other multi-dimensional data structures.

   Syntax:

   ```
   int[] arr = { 1, 2, 3, 4, 5 };
   ```

4. String

   In Java, a string is a sequence of characters. In simple words, we can define a string as an array of characters. The difference between a character array and a string in Java is that the string is designed to hold a sequence of characters in a single variable, whereas a character array is a collection of separate char-type entities. Note that, unlike C/C++, Java strings are not terminated with a null character.

   Syntax:

   ```
   // string without using new operator
   String s = "tpointtech";
   // String using new operator
   String str = new String("Austria");
   ```

5. enum

Java also includes other non-primitive data types, such as enums and collections. Enums are used to define a set of named constants, providing a way to represent a fixed set of values. Collections are a framework of classes and interfaces that provide dynamic data structures such as lists, sets, and maps, which can grow or shrink in size as needed.

Syntax:

```
enum Grade {
  FIRST,
  SECOND,
  THIRD
}
```

Java Non-Primitive Data Type Example

```
enum Color {
    RED,
    GREEN,
    BLUE;
}
public class Main {
    public static void main(String[] args) {
        String str = "Hello";
        int[] arr = { 1, 2, 3, 4, 5 };
        Color clr = Color.BLUE;
        System.out.println(clr);
        System.out.println(str);
        for (int member: arr) {
        System.out.print(member+", ");
        }
    }
}
```

Java Operators

Java operators are the symbols that are used to perform various operations on variables and values. By using these operators, we can perform operations like addition, subtraction, checking less than or greater than, etc.

There are different types of operators in Java, we have listed them below –

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Misc Operators

Java Arithmetic Operators

Arithmetic operators are used in mathematical expressions in the same way that they are used in algebra. The following table lists the arithmetic operators −

Assume integer variable A holds 10 and variable B holds 20, then –

Operator	Description	Example
+ (Addition)	Adds values on either side of the operator.	A + B will give 30
- (Subtraction)	Subtracts right-hand operand from left-hand operand.	A - B will give -10
* (Multiplication)	Multiplies values on either side of the operator.	A * B will give 200
/ (Division)	Divides left-hand operand by right-hand operand.	B / A will give 2
% (Modulus)	Divides left-hand operand by right-hand operand and returns remainder.	B % A will give 0
++ (Increment)	Increases the value of operand by 1.	B++ gives 21
-- (Decrement)	Decreases the value of operand by 1.	B-- gives 19

Example

The following example demonstrates the usage of arithmetic operators in Java:

```
public class ArithmeticExample {
   public static void main(String[] args) {
      int a = 10, b = 5;

      // Addition
      System.out.println("a + b = " + (a + b)); // 15
```

```
        // Subtraction
        System.out.println("a - b = " + (a - b)); // 5

        // Multiplication
        System.out.println("a * b = " + (a * b)); // 50

        // Division
        System.out.println("a / b = " + (a / b)); // 2

        // Modulus
        System.out.println("a % b = " + (a % b)); // 0
    }
}
```

When the above code is compiled and executed, it produces the following result –

```
a + b = 15
a - b = 5
a * b = 50
a / b = 2
a % b = 0
```

Java Assignment Operators

Assignment Operators are used to assign values to variables. These operators modify the value of a variable based on the operation performed. The most commonly used assignment operator is =, but Java provides multiple compound assignment operators for shorthand operations.

Following are the assignment operators supported by Java language –

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand.	C = A + B will assign value of A + B into C
+=	Add AND assignment operator. It adds right operand to the left operand and assign the result to left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts right operand from the left operand and assign the result to left operand.	C -= A is equivalent to C = C

		− A
*=	Multiply AND assignment operator. It multiplies right operand with the left operand and assign the result to left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator. It divides left operand with the right operand and assign the result to left operand.	C /= A is equivalent to C = C / A
%=	Modulus AND assignment operator. It takes modulus using two operands and assign the result to left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
|=	bitwise inclusive OR and assignment operator.	C |= 2 is same as C = C | 2

Example

The following example demonstrates the usage of assignment operators in Java:

```
public class AssignmentExample {
   public static void main(String[] args) {
      int a = 10;

      // Assign and add
      a += 5;
```

```
        System.out.println("a += 5: " + a); // 15

        // Assign and subtract
        a -= 3;
        System.out.println("a -= 3: " + a); // 12

        // Assign and multiply
        a *= 2;
        System.out.println("a *= 2: " + a); // 24

        // Assign and divide
        a /= 4;
        System.out.println("a /= 4: " + a); // 6

        // Assign and modulus
        a %= 5;
        System.out.println("a %= 5: " + a); // 1
    }
}
```

When the above code is compiled and executed, it produces the following result –

```
a += 5: 15
a -= 3: 12
a *= 2: 24
a /= 4: 6
a %= 5: 1
```

Java Relational Operators

Relational operators are used to compare two values. These operators return a boolean result: true if the condition is met and false otherwise. Relational operators are commonly used in decision-making statements like if conditions and loops.

There are following relational operators supported by Java language.

Assume variable A holds 10 and variable B holds 20, then –

Operator	Description	Example
== (equal to)	Checks if the values of two operands are equal or not, if yes then condition becomes true.	(A == B) is not true.
!= (not equal to)	Checks if the values of two operands are equal or not, if values are not equal then condition becomes true.	(A != B) is true.
> (greater than)	Checks if the value of left operand is greater than the value of right operand, if yes then	(A > B) is not true.

	condition becomes true.	
< (less than)	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	(A < B) is true.
>= (greater than or equal to)	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	(A >= B) is not true.
<= (less than or equal to)	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	(A <= B) is true.

Example
The following example demonstrates the usage of relational operators in Java:

```
public class RelationalExample {
   public static void main(String[] args) {
      int A = 10, B = 5;

      System.out.println("A == B: " + (A == B)); // false
      System.out.println("A != B: " + (A != B)); // true
      System.out.println("A > B: " + (A > B));   // true
      System.out.println("A < B: " + (A < B));   // false
      System.out.println("A >= B: " + (A >= B)); // true
      System.out.println("A <= B: " + (A <= B)); // false
   }
}
```

When the above code is compiled and executed, it produces the following result –

```
A == B: false
A != B: true
A > B: true
A < B: false
A >= B: true
A <= B: false
```

Java Logical Operators
Logical operators are used to perform logical operations on boolean values. These operators are commonly used in decision-making statements such as if conditions and loops to control program flow.

The following table lists the logical operators −

Assume Boolean variables A holds true and variable B holds false, then –

Operator	Description	Example
&& (logical and)	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false
|| (logical or)	Called Logical OR Operator. If any of the two operands are non-zero, then the condition becomes true.	(A || B) is true
! (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B) is true

Example
The following example demonstrates the usage of logical operators in Java:

```
public class LogicalExample {
   public static void main(String[] args) {
      boolean A = true, B = false;

      System.out.println("A && B: " + (A && B)); // false
      Syst
em.out.println("A || B: " + (A || B)); // true
      System.out.println("!A: " + (!A)); // false
      System.out.println("!B: " + (!B)); // true
   }
}
```

When the above code is compiled and executed, it produces the following result –

```
A && B: false
A || B: true
!A: false
!B: true
```

Java Bitwise Operators
Bitwise operators are used to perform operations at the binary (bit) level. These operators work on individual bits of numbers. They are commonly used in low-level programming, encryption, and performance optimization.

Java defines several bitwise operators, which can be applied to the integer types, long, int, short, char, and byte.

Bitwise operator works on bits and performs bit-by-bit operation. Assume if a = 60 and b = 13; now in binary format they will be as follows –

a = 0011 1100

b = 0000 1101

a&b = 0

a^b = 0011 0001

~a = 1100 0011000 1100

a|b = 0011 1101

The following table lists the bitwise operators −

Assume integer variable A holds 60 and variable B holds 13 then –

Operator	Description	Example
& (bitwise and)	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) will give 12 which is 0000 1100
| (bitwise or)	Binary OR Operator copies a bit if it exists in either operand.	(A | B) will give 61 which is 0011 1101
^ (bitwise XOR)	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) will give 49 which is 0011 0001
~ (bitwise compliment)	Binary Ones Complement Operator is unary and has the effect of 'flipping' bits.	(~A) will give -61 which is 1100 0011 in 2's complement form due to a signed binary number.
<< (left shift)	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 will give 240 which is 1111 0000
>> (right shift)	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 will give 15 which is 1111
>>> (zero fill right shift)	Shift right zero fill operator. The left operands value is moved right by the number of bits specified by the right operand and shifted values are filled up with zeros.	A >>>2 will give 15 which is 0000 1111

Example

The following example demonstrates the usage of bitwise operators in Java:

```
public class BitwiseExample {
   public static void main(String[] args) {
      int A = 60; // 0011 1100
      int B = 13; // 0000 1101

      System.out.println("A & B: " + (A & B)); // 12  (0000 1100)
      System.out.println("A | B: " + (A | B)); // 61  (0011 1101)
      System.out.println("A ^ B: " + (A ^ B)); // 49  (0011 0001)
      System.out.println("~A: " + (~A)); // -61  (1100 0011 in 2's complement)
      System.out.println("A << 2: " + (A << 2)); // 240 (1111 0000)
      System.out.println("A >> 2: " + (A >> 2)); // 15  (0000 1111)
      System.out.println("A >>> 2: " + (A >>> 2)); // 15  (0000 1111)
   }
}
```

When the above code is compiled and executed, it produces the following result –

```
A & B: 12
A | B: 61
A ^ B: 49
~A: -61
A << 2: 240
A >> 2: 15
A >>> 2: 15
```

Java Miscellaneous Operators

There are few other operators supported by Java Language.

Conditional Operator (? :)

Conditional operator is also known as the ternary operator. This operator consists of three operands and is used to evaluate Boolean expressions. The goal of the operator is to decide, which value should be assigned to the variable. The operator is written as –

variable x = (expression) ? value if true : value if false

Example

Following is an example −

In this example, we're creating two variables a and b and using ternary operator we've decided the values of b and printed it.

```
public class Test {

   public static void main(String args[]) {
      int a, b;
      a = 10;
      b = (a == 1) ? 20: 30;
      System.out.println( "Value of b is : " +  b );
```

```
    b = (a == 10) ? 20: 30;
    System.out.println( "Value of b is : " + b );
  }
}
```

Output

Value of b is : 30

Value of b is : 20

instanceof Operator

This operator is used only for object reference variables. The operator checks whether the object is of a particular type (class type or interface type). instanceof operator is written as –

(Object reference variable) instanceof (class/interface type)

If the object referred by the variable on the left side of the operator passes the IS-A check for the class/interface type on the right side, then the result will be true. Following is an example –

Example

In this example, we're creating a String variable name and then using instanceof operator we've checking the name is of String or not.

```
public class Test {

  public static void main(String args[]) {

    String name = "James";

    // following will return true since name is type of String
    boolean result = name instanceof String;
    System.out.println( result );
  }
}
```

Output

True

This operator will still return true, if the object being compared is the assignment compatible with the type on the right. Following is one more example –

Example

In this example, we're creating a variable a of class Vehicle and then using instanceof operator we've checking the name is of type Car or not.

```
class Vehicle {}
```

```
public class Car extends Vehicle {

   public static void main(String args[]) {

      Vehicle a = new Car();
      boolean result =  a instanceof Car;
      System.out.println( result );
   }
}
```

Output

True

Java Operators Precedence & Associativity

Operator precedence determines the grouping of terms in an expression. This affects how an expression is evaluated. Certain operators have higher precedence than others; for example, the multiplication operator has higher precedence than the addition operator −

For example, x = 7 + 3 * 2; here x is assigned 13, not 20 because operator * has higher precedence than +, so it first gets multiplied with 3 * 2 and then adds into 7.

Here, operators with the highest precedence appear at the top of the table, those with the lowest appear at the bottom. Within an expression, higher precedence operators will be evaluated first.

Category	Operator	Associativity
Postfix	expression++ expression--	Left to right
Unary	++expression --expression +expression -expression ~ !	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >> >>>	Left to right
Relational	< > <= >= instanceof	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right

Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= ^= |= <<= >>= >>>=	Right to left

Example

The following example demonstrates how operator precedence and associativity affect expression evaluation in Java:

```
public class OperatorPrecedenceExample {
   public static void main(String[] args) {
      int result1 = 10 + 5 * 2; // Multiplication (*) has higher precedence than addition (+)
      int result2 = (10 + 5) * 2; // Parentheses () change precedence
      int result3 = 20 / 4 * 2; // Left-to-right associativity
      int result4 = 10 - 3 + 2; // Left-to-right associativity

      System.out.println("10 + 5 * 2 = " + result1);
      System.out.println("(10 + 5) * 2 = " + result2);
      System.out.println("20 / 4 * 2 = " + result3);
      System.out.println("10 - 3 + 2 = " + result4);
   }
}
```

When the above code is compiled and executed, it produces the following result –

```
10 + 5 * 2 = 20
(10 + 5) * 2 = 30
20 / 4 * 2 = 10
10 - 3 + 2 = 9
```

Java Keywords

In Java, keywords are the reserved words that have some predefined meanings and are used by the Java compiler for some internal process or represent some predefined actions. These words cannot be used as identifiers such as variable names, method names, class names, or object names.

Java Keywords List

Keywords	Usage
abstract	Specifies that a class or method will be implemented later, in a subclass
assert	Assert describes a predicate placed in a Java program to indicate that the developer thinks that the predicate is always true at that place.
boolean	A data type that can hold True and False values only
break	A control statement for breaking out of loops.
byte	A data type that can hold 8-bit data values
case	Used in switch statements to mark blocks of text
catch	Catches exceptions generated by try statements
char	A data type that can hold unsigned 16-bit Unicode characters
class	Declares a new class
const	Reserved but not used
continue	Sends control back outside a loop
default	Specifies the default block of code in a switch statement
do	Starts a do-while loop
double	A data type that can hold 64-bit floating-point numbers
else	Indicates alternative branches in an if statement
enum	A Java keyword is used to declare an enumerated

Keywords	Usage
	type. Enumerations extend the base class.
extends	Indicates that a class is derived from another class or interface
final	Indicates that a variable holds a constant value or that a method will not be overridden
finally	Indicates a block of code in a try-catch structure that will always be executed
float	A data type that holds a 32-bit floating-point number
for	Used to start a for loop
goto	Reserved but not used
if	Tests a true/false expression and branches accordingly
implements	Specifies that a class implements an interface
import	References other classes
instanceof	Indicates whether an object is an instance of a specific class or implements an interface
int	A data type that can hold a 32-bit signed integer
interface	Declares an interface
long	A data type that holds a 64-bit integer
native	Specifies that a method is implemented with native (platform-specific) code
new	Creates new objects

Keywords	Usage
null	This indicates that a reference does not refer to anything
package	Declares a Java package
private	An access specifier indicating that a method or variable may be accessed only in the class it's declared in
protected	An access specifier indicating that a method or variable may only be accessed in the class it's declared in (or a subclass of the class it's declared in or other classes in the same package)
public	An access specifier used for classes, interfaces, methods, and variables indicating that an item is accessible throughout the application (or where the class that defines it is accessible)
return	Sends control and possibly a return value back from a called method
short	A data type that can hold a 16-bit integer
static	Indicates that a variable or method is a class method (rather than being limited to one particular object)
strictfp	A Java keyword is used to restrict the precision and rounding of floating-point calculations to ensure portability.
super	Refers to a class's base class (used in a method or class constructor)
switch	A statement that executes code based on a test value
synchronized	Specifies critical sections or methods in multithreaded code

Keywords	Usage
this	Refers to the current object in a method or constructor
throw	Creates an exception
throws	Indicates what exceptions may be thrown by a method
transient	Specifies that a variable is not part of an object's persistent state
try	Starts a block of code that will be tested for exceptions
void	Specifies that a method does not have a return value
volatile	This indicates that a variable may change asynchronously
while	Starts a while loop
sealed	The sealed keyword is used to declare a class as "sealed," meaning it restricts which classes can extend it.
permits	The permits keyword is used within a sealed class declaration to specify the subclasses that are permitted to extend it.

Important Points:

- The keywords const and goto are reserved, even though they are not currently used in Java.
- true, false, and null look like keywords, but in actuality they are literals. However, they still can't be used as identifiers in a program.
- In Java, keywords are case-sensitive, and writing Java keywords in upper case (like IF instead of if) will throw an error.

Java Identifiers

An identifier in Java is the name given to Variables, Classes, Methods, Packages, Interfaces, etc. These are the unique names used to identify programming elements. Every Java Variable must be identified with a unique name.

Example:

```
public class Test
{
   public static void main(String[] args)
   {
      int a = 20;
   }
}
```

In the above Java code, we have 5 identifiers as follows:
Test: Class Name
main: Method Name
String: Predefined Class Name
args: Variable Name
a: Variable Name

Rules For Naming Java Identifiers

There are certain rules for defining a valid Java identifier. These rules must be followed, otherwise, we get a compile-time error. These rules are also valid for other languages like C and C++.

- The only allowed characters for identifiers are all alphanumeric characters([A-Z],[a-z],[0-9]), '$'(dollar sign) and '_' (underscore). For example, "geek@" is not a valid Java identifier as it contains a '@', a special character.
- Identifiers should not start with digits([0-9]). For example, "123geeks" is not a valid Java identifier.
- Java identifiers are case-sensitive.
- There is no limit on the length of the identifier, but it is advisable to use an optimum length of 4 – 15 letters only.
- Reserved Words can't be used as an identifier. For example, "int while = 20;" is an invalid statement as a while is a reserved word.

Note: Java has 53 reserved words (including 50 keywords and 3 literals), that are not allowed to be used as identifiers.

Reserved Words in Java

Any programming language reserves some words to represent functionalities defined by that language. These words are called reserved words. They can be briefly categorized into two parts:
Keywords (50): Keywords define functionalities.

literals (3): Literals define value.
Identifiers are stored by symbol tables and used during the lexical, syntax, and semantic analysis phase of compilation.
List of Java Reserved Words

abstract	continue	for	protected	transient
Assert	Default	Goto	public	Try
Boolean	Do	If	Static	throws
break	double	implements	strictfp	Package
byte	else	import	super	Private
case	enum	Interface	Short	switch
Catch	Extends	instanceof	return	void
Char	Final	Int	synchronized	volatile
class	finally	long	throw	Date
const	float	Native	This	while

What is a Constant and How to Define Constants in Java
A constant is a variable whose value cannot change once it has been assigned. Java doesn't have built-in support for constants.

A constant can make our program more easily read and understood by others. In addition, a constant is cached by the JVM as well as our application, so using a constant can improve performance.

To define a variable as a constant, we just need to add the keyword "final" in front of the variable declaration.

Syntax
final float pi = 3.14f;
The above statement declares the float variable "pi" as a constant with a value of 3.14f. We cannot change the value of "pi" at any point in time in the program. Later if we try to do that

by using a statement like "pi=5.25f", Java will throw errors at compile time itself. It is not mandatory that we need to assign values of constants during initialization itself.

In the below example, we can define the primitive data type (byte, short, int, long, float, double, boolean and char) variables as constants by just adding the keyword "final" when we declare the variable.

Example

```
public class ConstantsDemo {
  public static void main(String args[]) {
    final byte var1 = 2;
    final byte var2;
    var2 = -3;
    final short var3 = 32;
    final short var4;
    var4 = -22;
    final int var5 = 100;
    final int var6;
    var6 = -112;
    final long var7 = 20000;
    final long var8;
    var8 = -11223;
    final float var9 = 21.23f;
    final float var10;
    var10 = -121.23f;
    final double var11 = 20000.3223;
    final double var12;
    var12 = -11223.222;
    final boolean var13 = true;
    final boolean var14;
    var14 = false;
    final char var15 = 'e';
    final char var16;
    var16 = 't';
   // Displaying values of all variables
    System.out.println("value of var1 : "+var1);
    System.out.println("value of var2 : "+var2);
    System.out.println("value of var3 : "+var3);
    System.out.println("value of var4 : "+var4);
    System.out.println("value of var5 : "+var5);
    System.out.println("value of var6 : "+var6);
    System.out.println("value of var7 : "+var7);
    System.out.println("value of var8 : "+var8);
```

```
    System.out.println("value of var9 : "+var9);
    System.out.println("value of var10 : "+var10);
    System.out.println("value of var11 : "+var11);
    System.out.println("value of var12 : "+var12);
    System.out.println("value of var13 : "+var13);
    System.out.println("value of var14 : "+var14);
    System.out.println("value of var15 : "+var15);
    System.out.println("value of var16 : "+var16);
  }
}
```

Output

```
value of var1 : 2
value of var2 : -3
value of var3 : 32
value of var4 : -22
value of var5 : 100
value of var6 : -112
value of var7 : 20000
value of var8 : -11223
value of var9 : 21.23
value of var10 : -121.23
value of var11 : 20000.3223
value of var12 : -11223.222
value of var13 : true
value of var14 : false
value of var15 : e
value of var16 : t
```

Java Variables

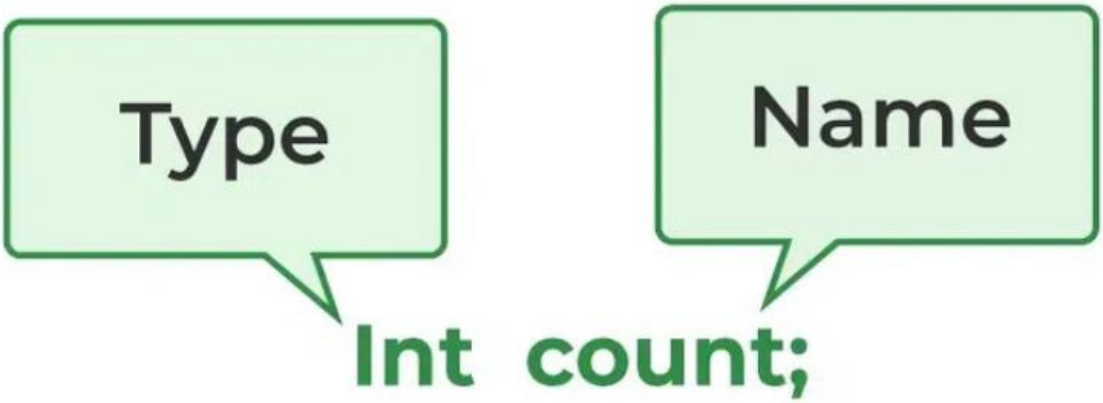
Variables are the containers for storing the data values or you can also call it a memory location name for the data. Every variable has a:

Data Type – The kind of data that it can hold. For example, int, string, float, char, etc.
Variable Name – To identify the variable uniquely within the scope.
Value – The data assigned to the variable.
There are three types of variables in Java – Local, Instance, and Static.

Example:

```
int age = 27;                // integer variable having value 27

String name = "gfg"     // string variable
```

How to Declare Java Variables?
We can declare variables in Java as pictorially depicted below:



From the image, it can be easily perceived that while declaring a variable, we need to take care of two things that are:

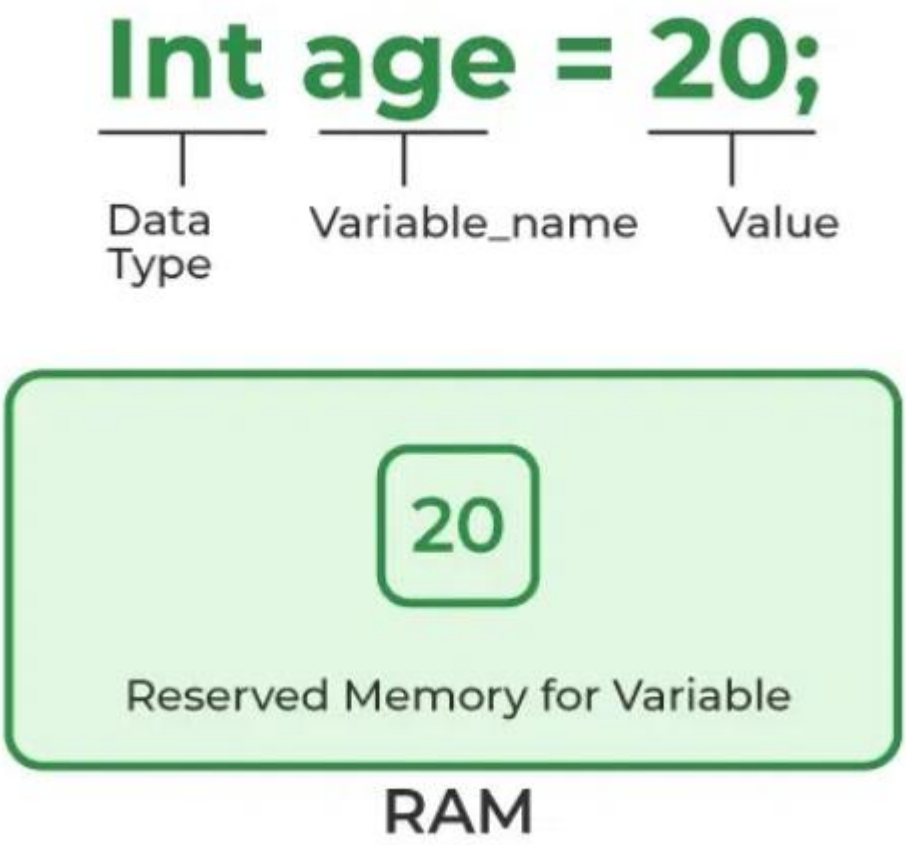
datatype: In Java, a data type define the type of data that a variable can hold.
data_name: Name was given to the variable.

In this way, a name can only be given to a memory location. It can be assigned values in two ways:

Variable Initialization
Assigning value by taking input

How to Initialize Java Variables?

It can be perceived with the help of 3 components explained above:



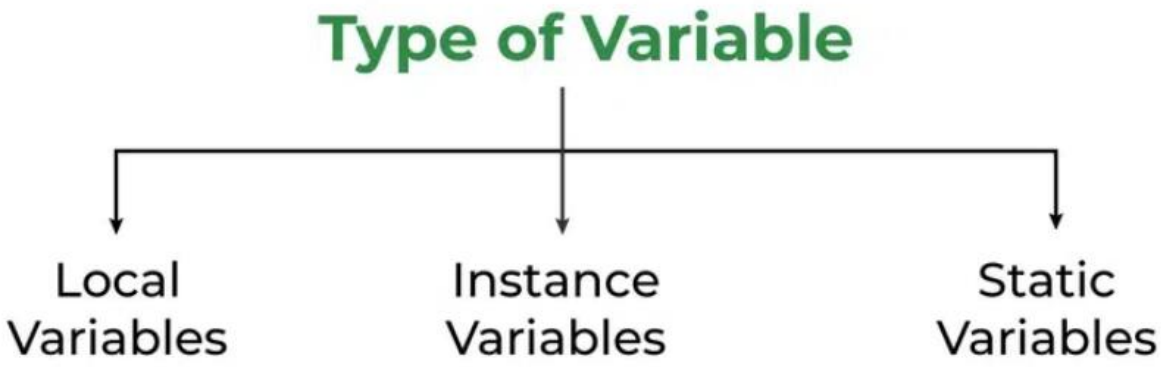
Example:

```
// Declaring float variable
float simpleInterest;

// Declaring and initializing integer variable
int time = 10, speed = 20;

// Declaring and initializing character variable
char var = 'h';
```

Types of Java Variables

Now let us discuss different types of variables which are listed as follows:

Local Variables
Instance Variables
Static Variables



1. Local Variables
   A variable defined within a block or method or constructor is called a local variable.

   - The Local variable is created at the time of declaration and destroyed after exiting from the block or when the call returns from the function.
   - The scope of these variables exists only within the block in which the variables are declared, i.e., we can access these variables only within that block.
   - Initialization of the local variable is mandatory before using it in the defined scope.

Example 1:

```
// Java Program to show the use of local variables
import java.io.*;

class GFG {
   public static void main(String[] args)
   {
      // Declared a Local Variable
      int var = 10;

      // This variable is local to this main method only
      System.out.println("Local Variable: " + var);
   }
}
```

Output
Local Variable: 10

2. Instance Variables
   Instance variables are non-static variables and are declared in a class outside of any method, constructor, or block.

- As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
- Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier, then the default access specifier will be used.
- Initialization of an instance variable is not mandatory. Its default value is dependent on the data type of variable. For String it is null, for float it is 0.0f, for int it is 0, for Wrapper classes like Integer it is null, etc.
- Scope of instance variables are throughout the class except the static contexts.
- Instance variables can be accessed only by creating objects.
- We initialize instance variables using constructors while creating an object. We can also use instance blocks to initialize the instance variables.

Example:

```
// Java Program to show the use of
// Instance Variables
import java.io.*;

class GFG {

    // Declared Instance Variable
    public String geek;
    public int i;
    public Integer I;
    public GFG()
    {
        // Default Constructor
        // initializing Instance Variable
        this.geek = "Shubham Jain";
    }

    // Main Method
    public static void main(String[] args)
    {
        // Object Creation
        GFG name = new GFG();

        // Displaying O/P
        System.out.println("Geek name is: " + name.geek);
        System.out.println("Default value for int is "+ name.i);

        // toString() called internally
        System.out.println("Default value for Integer is "+ name.I);
```

```
    }
}
```

Output

Geek name is: Shubham Jain

Default value for int is 0

Default value for Integer is null

3. Static Variables

Static variables are also known as class variables.

- These variables are declared similarly to instance variables. The difference is that static variables are declared using the static keyword within a class outside of any method, constructor, or block.
- Unlike instance variables, we can only have one copy of a static variable per class, irrespective of how many objects we create.
- Static variables are created at the start of program execution and destroyed automatically when execution ends.
- Initialization of a static variable is not mandatory. Its default value is dependent on the data type of variable. For String it is null, for float it is 0.0f, for int it is 0, for Wrapper classes like Integer it is null, etc.
- If we access a static variable like an instance variable (through an object), the compiler will show a warning message, which won't halt the program. The compiler will replace the object name with the class name automatically.
- If we access a static variable without the class name, the compiler will automatically append the class name. But for accessing the static variable of a different class, we must mention the class name as 2 different classes might have a static variable with the same name.
- Static variables cannot be declared locally inside an instance method.
- Static blocks can be used to initialize static variables.

Example:

```
// Java Program to show the use of
// Static variables
import java.io.*;

class GFG {
    // Declared static variable
    public static String geek = "Shubham Jain";

    public static void main(String[] args)
    {

        // geek variable can be accessed without object
        // creation Displaying O/P GFG.geek --> using the
        // static variable
```

```
        System.out.println("Geek Name is : " + GFG.geek);

        // static int c = 0;
        // above line, when uncommented,
        // will throw an error as static variables cannot be
        // declared locally.
    }
}
```

Output

Geek Name is : Shubham Jain

Instance Variables vs Static Variables

- Now let us discuss the differences between the Instance variables and the Static variables:
- Each object will have its own copy of an instance variable, whereas we can only have one copy of a static variable per class, irrespective of how many objects we create. Thus, static variables are good for memory management.
- Changes made in an instance variable using one object will not be reflected in other objects as each object has its own copy of the instance variable. In the case of a static variable, changes will be reflected in other objects as static variables are common to all objects of a class.
- We can access instance variables through object references, and static variables can be accessed directly using the class name.
- Instance variables are created when an object is created with the use of the keyword 'new' and destroyed when the object is destroyed. Static variables are created when the program starts and destroyed when the program stops.

Syntax: Static and instance variables

```
class GFG
{
    // Static variable
    static int a;

    // Instance variable
    int b;
}
```

Typecasting in Java

Typecasting in Java is the process of converting one data type to another data type using the casting operator. When you assign a value from one primitive data type to another type, this is known as type casting. To enable the use of a variable in a specific manner, this method requires explicitly instructing the Java compiler to treat a variable of one data type as a variable of another data type.

Syntax:

<datatype> variableName = (<datatype>) value;

Types of Type Casting

There are two types of Type Casting in java:

- Widening Type Casting
- Narrow Type Casting

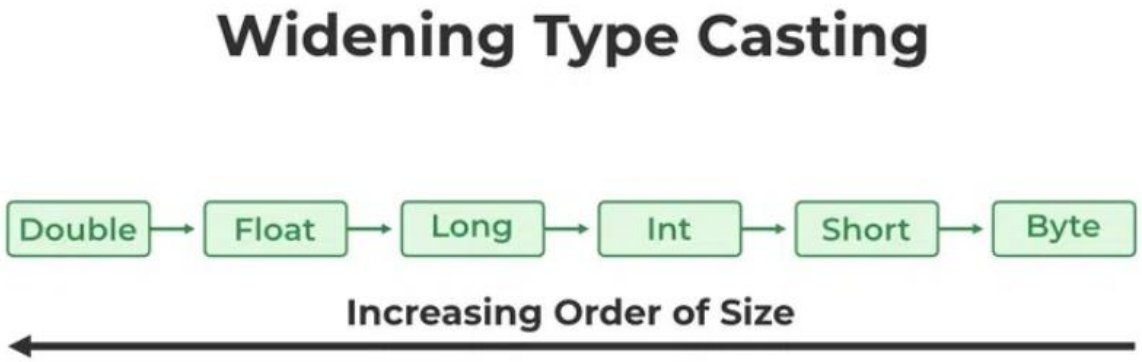
Widening Type Casting

A lower data type is transformed into a higher one by a process known as widening type casting. Implicit type casting and casting down are some names for it. It occurs naturally. Since there is no chance of data loss, it is secure. Widening Type casting occurs when:

- The target type must be larger than the source type.
- Both data types must be compatible with each other.

Syntax:

larger_data_type variable_name = smaller_data_type_variable;



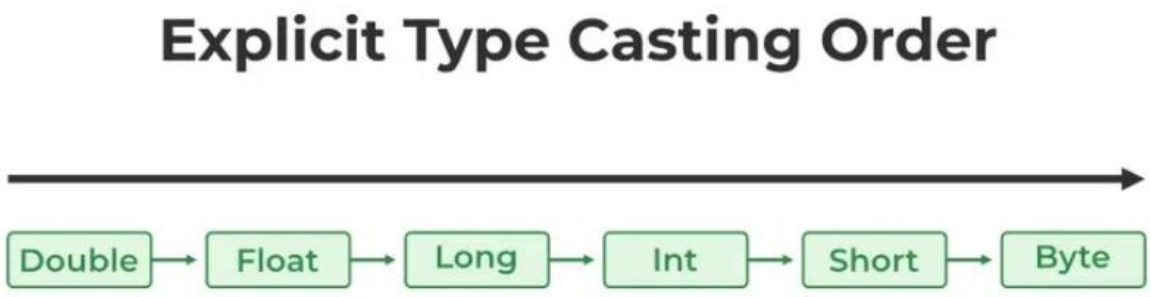

```
// Java program to demonstrate Widening TypeCasting
import java.io.*;

class GFG {
   public static void main(String[] args)
   {
      int i = 10;
```

```
        // Wideing TypeCasting (Automatic Casting)
        // from int to long
        long l = i;

        // Wideing TypeCasting (Automatic Casting)
        // from int to double
        double d = i;

        System.out.println("Integer: " + i);
        System.out.println("Long: " + l);
        System.out.println("Double: " + d);
    }
}
```

Output
Integer: 10
Long: 10
Double: 10.0

Narrow Type Casting
The process of downsizing a bigger data type into a smaller one is known as narrowing type casting. Casting up or explicit type casting are other names for it. It doesn't just happen by itself. If we don't explicitly do that, a compile-time error will occur. Narrowing type casting is unsafe because data loss might happen due to the lower data type's smaller range of permitted values. A cast operator assists in the process of explicit casting.
Syntax:

smaller_data_type variable_name = (smaller_data_type) larger_data_type_variable;



Example:
```
// Java Program to demonstrate Narrow type casting
import java.io.*;

class GFG {
    public static void main(String[] args)
```

```
    {
        double i = 100.245;

        // Narrowing Type Casting
        short j = (short)i;
        int k = (int)i;

        System.out.println("Original Value before Casting"
                        + i);
        System.out.println("After Type Casting to short "
                        + j);
        System.out.println("After Type Casting to int "
                        + k);
    }
}
```

Output
Original Value before Casting100.245
After Type Casting to short 100
After Type Casting to int 100

Types of Explicit Casting
Mainly there are two types of Explicit Casting:

- Explicit Upcasting
- Explicit Downcasting

Explicit Upcasting
Upcasting is the process of casting a subtype to a supertype in the inheritance tree's upward direction. When a sub-class object is referenced by a superclass reference variable, an automatic process is triggered without any further effort.

```
// Java Program to demonstrate Explicit Upcasting
import java.io.*;

class Animal {
    public void makeSound()
    {
        System.out.println("The animal makes a sound");
    }
}

class Dog extends Animal {
    public void makeSound()
    {
        System.out.println("The dog barks");
    }
```

```
    public void fetch()
    {
        System.out.println("The dog fetches a ball");
    }
}
class GFG {
    public static void main(String[] args)
    { // Upcasting
        Animal animal = new Dog();
        // Calls the overridden method in Dog class
        animal.makeSound();
        // This would give a compile error as fetch() is not
        // a method in Animal class
        // animal.fetch();
    }
}
```

The dog barks

Explicit Downcasting

When a subclass type refers to an object of the parent class, the process is referred to as downcasting. If it is done manually, the compiler issues a runtime ClassCastException error. It can only be done by using the instanceof operator. Only the downcast of an object that has already been upcast is possible.

Example:

```
// Java Program to demonstrate Explicit downcasting
import java.io.*;
class Animal {
    public void eat()
    {
        System.out.println("The animal is eating.");
    }
}

class Cat extends Animal {
    public void meow()
    {
        System.out.println("The cat is meowing.");
    }
}

class GFG {
    public static void main(String[] args)
```

```
    {
        Animal animal = new Cat();
        animal.eat();

        // Explicit downcasting
        Cat cat = (Cat)animal;
        cat.meow();
    }
}
```

Output
The animal is eating.
The cat is meowing.

Three kinds of control structures in Java

1. **Control statements in java / Conditional Branches in java**

   Java control statements use to choose the path for execution. There are some types of control statements:

   if statement

   It is a simple decision-making statement. It uses to decide whether the statement or block of statements should be executed or not. A Block of statements executes only if the given condition returns true otherwise the block of the statement skips. The if statement always accepts the boolean value and performs the action accordingly. Here we will see the simplest example of an if statement.

```
public class IfStatementExample
{
    public static void main(String arg[]) {
        int a = 5;
        int b = 4;
        // Evaluating the expression that will return true or false
        if (a > b) {
            System.out.println("a is greater than b");
        }
    }
}
```

   Output: a is greater than b

   nested if statement

   The nested if statements mean an if statement inside another if statement. The inner block of the if statement executes only if the outer block condition is true. This statement is useful when you want to test a dependent condition. Here we will see the example of nested if with a common example and see how it works.

```
{
public static void main(String arg[])
{
int age = 20;
boolean hasVoterCard = true;
// Evaluating the expression that will return true or false
if (age > 18)
{
// If outer condition is true then this condition will be check
if (hasVoterCard)
{
System.out.println("You are Eligible");
}
}
}
}
```

Output: You are Eligible

if-else statement

In the if statement, the block of statements executes when the given condition returns true otherwise block of the statement skips. Suppose when we want to execute the same block of code if the condition is false. Here we come on the if-else statement. In an if-else statement, there are two blocks one is an if block and another is an else block. If a certain condition is true, then if block executes otherwise else block executes. Let’s see the example and understand it.

```
public class IfElseStatementExample
{
   public static void main(String arg[]) {
      int a = 10;
      int b = 50;
      // Evaluating the expression that will return true or false
      if (a > b)
      {
         System.out.println("a is greater than b");
      }
      else
      {
         System.out.println("b is greater than a");
      }
   }
}
```

Output: b is greater than a

if-else if statement/ ladder if statements

If we want to execute the different codes based on different conditions then we can use if-else-if. It is known as if-else if ladder. This statement executes from the top down. During the execution of conditions if any condition is found true, then the statement associated with that if statement is executed, and the rest of the code will be skipped. If none of the conditions returns true, then the final else statement executes.

```
public class IfElseIfStatementExample
{
   public static void main(String arg[]) {
      int a = 10;
      // Evaluating the expression that will return true or false
      if (a == 1)
      {
         System.out.println("Value of a: "+a);
      }
      // Evaluating the expression that will return true or false
      else if(a == 5)
      {
         System.out.println("Value of a: "+a);
      }
      // Evaluating the expression that will return true or false
      else if(a == 10)
      {
         System.out.println("Value of a: "+a);
      }
      else
      {
         System.out.println("else block");
      }
   }
}
```

Switch statement

The switch statement in java is like the if-else-if ladder statement. To reduce the code complexity of the if-else-if ladder switch statement comes.

In a switch, the statement executes one from multiple statements based on condition. In the switch statements, we have a number of choices and we can perform a different task for each choice.

```
public class SwitchStatementExample
```

```
{
    public static void main(String arg[]) {
        int a = 10;
        // Evaluating the expression that will return true or false
        switch(a)
        {
            case 1:
                System.out.println("Value of a: 1");
                break;
            case 5:
                System.out.println("Value of a: 5");
                break;
            case 10:
                System.out.println("Value of a: 10");
                break;
            default:
                System.out.println("else block");
                break;
        }
    }
}
```

Output: Value of a: 10

2. Loops in Java / Looping statements in java

The loop in java uses to iterate the instruction multiple times. When we want to execute the statement a number of times then we use loops in java.

for loop

A for loop executes the block of code several times. It means the number of iterations is fixed. JAVA provides a concise way of writing the loop structure. When we want to execute the block of code serval times,

```
public class ForLoopExample
{
    public static void main(String arg[])
    {
        // Creating a for loop
        for(int i = 1; i <= 3; i++)
        {
            System.out.println("Iteration number: "+i);
        }
    }
```

```
}
```

Output: Iteration number: 1
Iteration number: 2
Iteration number: 3

while loop

A while loop allows code to execute repeatedly until the given condition returns true. If the number of iterations doesn't fix, it recommends using a while loop. It is also known as an entry control loop

```
public class WhileLoopExample
{
   public static void main(String arg[])
   {
      // Creating a while loop
      int i = 1;
      while(i <= 3)
      {
         System.out.println("Iteration number: "+i);
         i++;
      }
   }
}
```

Output: Iteration number: 1
Iteration number: 2
Iteration number: 3

do-while loop

It is like a while loop with the only difference being that it checks for the condition after the execution of the body of the loop. It is also known as Exit Control Loop.

```
public class DoWhileLoopExample
{
   public static void main(String arg[])
   {
      // Creating a while loop
      int i = 1;
      do
      {
         System.out.println("Iteration number: "+i);
```

```
            i++;
        }
        while(i <= 3);
    }
}
```

Output: Iteration number: 1
Iteration number: 2
Iteration number: 3

3. Branching Statements in Java
   These use to alter the flow of control in loops.

   Break

   If a break statement encounters inside a loop, the loop immediately terminates, and the control of the program goes to the next statement following the loop. It is used along with the if statement in loops. If the condition is true, then the break statement terminates the loop immediately.

```
public class BreakStatementExample
{
    public static void main(String arg[])
    {
        for(int i = 0; i <=3; i++)
        {
            System.out.println("Iteration number:" + i);
            if(i == 2)
                break;
        }
    }
}
```

Output: Iteration number:0
Iteration number:1
Iteration number:2

Continue

The continue statement uses in a loop control structure. If you want to skip some code and need to jump to the next iteration of the loop immediately. Then you can use a continue statement. It can be used in for loop or a while loop.

```
public class ContinueStatementExample
{
```

```
    public static void main(String arg[])
    {
        for(int i = 0; i <=3; i++)
        {
            if(i == 1)
                continue;
            System.out.println("Iteration number:" + i);
        }
    }
}
```

Output: Iteration number:0
Iteration number:2
Iteration number:3